Quantum Physics-Informed Neural Networks

Solving PDEs and ODEs with Variational Quantum Circuits

Morten Hjorth-Jensen

Department of Physics & CCSE
University of Oslo

2026

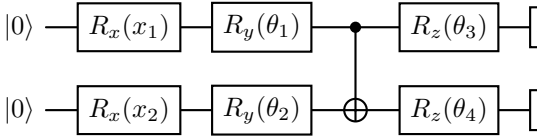
Outline

QNN as a Variational Quantum State (from week15.tex)

The central object of a **Quantum Neural Network**:

$$|\psi(x,\theta)\rangle = U(\theta)\,U(x)\;|0\rangle^{\otimes n}$$

$$f(x,\theta) = \langle\psi(x,\theta)|\;O\;|\psi(x,\theta)\rangle$$



- $U(x)$: **feature map** — encodes the input
- $U(\theta)$: **variational ansatz** — trainable
- O: **observable** — defines the output scalar

Pauli ansatz

$$U(\theta) = e^{-iH(\theta)}, \quad H(\theta) = \sum_{\alpha} \theta_\alpha P_\alpha$$

Key tool: parameter-shift rule

$$\frac{\partial f}{\partial \theta} = \tfrac{1}{2}\left[f(\theta + \tfrac{\pi}{2}) - f(\theta - \tfrac{\pi}{2})\right]$$

Exact gradient, 2 circuit evaluations per parameter.

QFI, TDVP, and Natural Gradient

Quantum Fisher Information (QFI):

$$F_{ij} = 4\,\mathrm{Re}(\langle\partial_i\psi|\partial_j\psi\rangle - \langle\partial_i\psi|\psi\rangle\langle\psi|\partial_j\psi\rangle)$$

Riemannian metric on the variational manifold.

Time-Dependent Variational Principle (TDVP):

$$\sum_j F_{ij}\,\dot{\theta}_j = C_i, \quad C_i = \mathrm{Im}\langle\partial_i\psi|H|\psi\rangle$$

Natural gradient descent:

$$\dot{\theta} = F^{-1}\nabla_\theta\mathcal{L}$$

BCH expansion

$$U^\dagger O\,U = O + i[H,O] + \tfrac{i^2}{2}[H,[H,O]] + \cdots$$

Depth $\leftrightarrow$ hierarchy of correlations.

Barren plateaus

Deep random circuits:

$$\mathrm{Var}(\nabla_\theta\mathcal{L}) \sim 2^{-n}$$

Mitigation: problem-inspired shallow ansätze — exactly what QPINNs do.

From Discriminative QNNs to Physics-Informed QNNs

Classical discriminative QNN

- Input: data pair (x_k, y_k)
- Loss: $\mathcal{L} = \sum_k (f(x_k; \theta) - y_k)^2$
- Learns a mapping from observations
- No physical constraint enforced

Physics-informed QNN (QPINN)

- Input: collocation points $x_k \in \Omega$
- Loss: residual of a differential equation

$$\mathcal{L} = \|\mathcal{D}[f_\theta]\|^2 + \lambda \|\mathcal{B}[f_\theta]\|^2$$

- Learns the solution to a PDE / ODE
- Physics is baked into the loss function

Why quantum circuits for PDEs?

1. Potential exponential compression of solution space via entanglement
2. Natural encoding of wave-like solutions through rotation gates
3. Gradient information from parameter-shift rule maps directly onto PDE residuals
4. Hardware noise acts as a form of regularisation
5. Connection to quantum simulation of differential operators

Classical PINNs: Raissi, Perdikaris, Karniadakis (2019)

A classical PINN approximates the solution $u(x,t)$ of a PDE

$$\mathcal{N}[u](x,t) = 0, \quad (x,t) \in \Omega$$

by a neural network $u_\theta(x,t)$ trained to minimise:

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r}\sum_{k=1}^{N_r}\left|\mathcal{N}[u_\theta](x_k,t_k)\right|^2}_{\text{PDE residual loss } \mathcal{L}_r} + \lambda \underbrace{\frac{1}{N_b}\sum_{k=1}^{N_b}\left|u_\theta(x_k^b,t_k^b) - g_k\right|^2}_{\text{BC/IC loss } \mathcal{L}_b}$$

where $\{x_k\}$ are *collocation points* sampled in Ω and $\{x_k^b\}$ are boundary/initial condition points.

Automatic differentiation

Classical PINNs exploit automatic differentiation to evaluate $\mathcal{N}[u_\theta]$ exactly (no finite differences).

Examples

- Poisson: $-\nabla^2 u = f$
- Heat: $\partial_t u - \kappa \nabla^2 u = 0$
- Wave: $\partial_{tt} u - c^2 \nabla^2 u = 0$
- Burgers: $\partial_t u + u \partial_x u = \nu \partial_{xx} u$
- Schrödinger: $i\partial_t \psi = H\psi$

Key challenge

Spectral bias: classical NNs learn

PINN Loss Landscape and Gradient Flow

For a general differential operator $\mathcal{D}$ the residual at collocation point x_k is:

$$r_k(\theta) = \mathcal{D}[f_\theta](x_k)$$

The total loss and its gradient:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_k r_k^2(\theta), \quad \nabla_\theta \mathcal{L} = \frac{2}{N}\sum_k r_k \nabla_\theta r_k$$

The Neural Tangent Kernel (NTK) of the PINN governs training dynamics:

$$\Theta_{kl}(\theta) = \nabla_\theta f_\theta(x_k)^\top \nabla_\theta f_\theta(x_l)$$

Eigenvalues of Θ determine the convergence rate for each frequency component.

Failure modes of classical PINNs

1. **Spectral bias:** slow convergence for high-frequency solutions
2. **Ill-conditioning:** loss balance between $\mathcal{L}_r$ and $\mathcal{L}_b$
3. **Stiff equations:** chaotic loss landscape
4. **High dimensionality:** curse of dimensionality for high-d PDEs

Quantum motivation

QPINNs address points 1 and 4: entangled circuits have richer frequency spectra; quantum memory scales exponentially with qubits.

QPINN: The Quantum Ansatz for PDE Solutions

Replace the classical network $u_\theta(x)$ with a **variational quantum circuit output**:

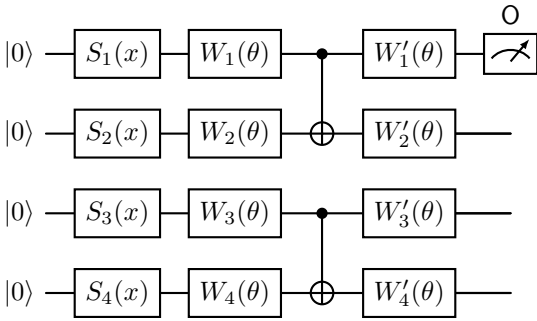
$$u_\theta(x) = \langle 0|\, U^\dagger(x,\theta)\, O\, U(x,\theta)\, |0\rangle$$

$$= \langle \psi(x,\theta)|\; O\; |\psi(x,\theta)\rangle$$

where:

- $U(x,\theta) = W_L(\theta)\cdots W_1(\theta)\, S(x)$
- $S(x)$: data-encoding unitary (feature map)
- $W_l(\theta)$: variational layers
- O: a fixed observable (e.g., Z_0)

The output $u_\theta(x) \in [-1,1]$ approximates the normalised PDE solution.



$S_i(x)$: encoding of coordinate x;
$W_i(\theta)$: variational Pauli rotations;
$O = Z_0$: observable giving scalar output.

Data Encoding Strategies for PDEs

The choice of encoding $x \mapsto S(x)$ is critical for QPINNs.

1. Angle encoding

$$S(x) = \bigotimes_{i=1}^{n} R_y(x\pi)$$

Simple, one qubit per input; output is a degree-n Fourier series in x.

2. Repeated angle encoding

$$S(x) = \prod_{l=1}^{L} \left[W_l(\theta) \bigotimes\nolimits_i R_y(x) \right]$$

Interleaving data re-uploads with variational layers (*data re-uploading*): extends the expressible frequency spectrum to $[-Ln/2, Ln/2]$.

3. IQP / ZZ encoding (richer correlations)

$$S(x) = e^{-ixH_{\text{enc}}}, \quad H_{\text{enc}} = \sum_i x\, Z_i + \sum_{i<j} x^2 Z_i Z_j$$

Fourier perspective (Schuld et al.)

Any QNN output is a partial Fourier series:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta)\, e^{i\omega x}$$

The set of accessible frequencies Ω is determined entirely by the data-encoding gates. Wider $\Omega \Rightarrow$ richer PDE solutions.

Derivatives of the Quantum Ansatz

The core challenge: PDE residuals require $\partial_x u_\theta$, $\partial_{xx} u_\theta$, etc. These are *derivatives with respect to the input* x, not the parameters θ.

Input-shift rule. For angle encoding $S(x) = e^{-ixP/2}$ (Pauli P):

$$\frac{\partial u_\theta}{\partial x} = \tfrac{1}{2}\big[u_\theta(x + \tfrac{\pi}{2}) - u_\theta(x - \tfrac{\pi}{2})\big]$$

This is the *same* parameter-shift formula, now applied to the **input encoding** gate.

Second derivative:

$$\frac{\partial^2 u_\theta}{\partial x^2} = \tfrac{1}{2}\big[u_\theta(x + \pi) - 2u_\theta(x) + u_\theta(x - \pi)\big]$$

General n-th order derivative via repeated shifts.

Key result

All spatial derivatives of $u_\theta(x)$ are computable by evaluating the *same circuit* at shifted input values — no ancilla qubits, no additional circuit depth.

Multi-dimensional PDE

For $u(x_1, \ldots, x_d)$ with n_i qubits encoding x_i:

$$\partial_{x_i} \partial_{x_j} u_\theta = \text{shift in } x_i \text{ and } x_j$$

Cross-derivatives come from shifts in *two* inputs simultaneously.

The QPINN Loss Function

For a PDE $\mathcal{D}[u](x) = f(x)$ on Ω with boundary conditions $u(x) = g(x)$ on $\partial\Omega$:

Total QPINN loss:

$$\mathcal{L}(\theta) = \mathcal{L}_r(\theta) + \lambda\,\mathcal{L}_b(\theta)$$

$$\mathcal{L}_r = \frac{1}{N_r}\sum_{k=1}^{N_r}\left|\mathcal{D}[u_\theta](x_k) - f(x_k)\right|^2$$

$$\mathcal{L}_b = \frac{1}{N_b}\sum_{k=1}^{N_b}\left|u_\theta(x_k^b) - g(x_k^b)\right|^2$$

where $u_\theta(x) = \langle\psi(x,\theta)|\,O\,|\psi(x,\theta)\rangle$ and all derivatives entering $\mathcal{D}[u_\theta]$ are evaluated via input-shift rules.

Gradient w.r.t. θ_μ

$$\frac{\partial\mathcal{L}_r}{\partial\theta_\mu} = \frac{2}{N_r}\sum_k r_k\,\frac{\partial r_k}{\partial\theta_\mu}$$

where $r_k = \mathcal{D}[u_\theta](x_k) - f(x_k)$ and

$$\frac{\partial r_k}{\partial\theta_\mu} = \mathcal{D}\left[\frac{\partial u_\theta}{\partial\theta_\mu}\right](x_k)$$

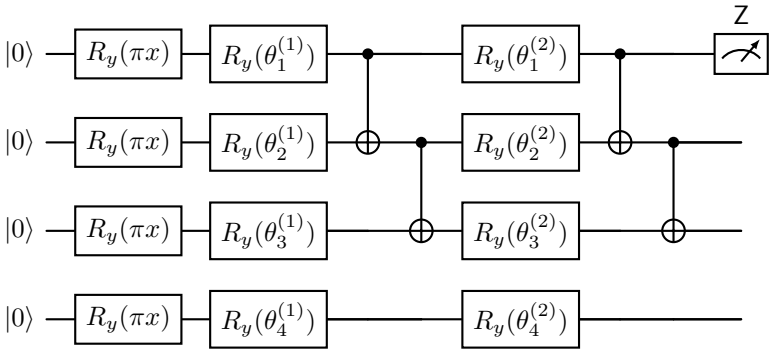
The operator $\mathcal{D}$ is linear in $u_\theta \Rightarrow$ shifts and $\mathcal{D}$ commute.

Parameter-shift + input-shift

Both types of derivatives are computable with $O(N_p \cdot N_r)$ circuit evaluations.

QPINN Circuit for a 1D PDE

Complete QPINN circuit for $-\partial_{xx}u = f(x)$, $x \in [0,1]$, with $n = 4$ qubits:



Evaluation for residual:

1. $u_\theta(x)$: run circuit with input x
2. $u_\theta(x \pm \frac{\pi}{2})$: two shifted circuits for $\partial_{xx}u_\theta$ via second-order shift

Gradient for training:

1. For each θ_μ: parameter-shift at x, $x \pm \frac{\pi}{2}$
2. Total: $2N_p \times 3N_r$ circuit evaluations per gradient step

The 1D Poisson Equation

Problem statement:

$$-\frac{d^2u}{dx^2} = f(x), \quad x \in (0,1)$$

$$u(0) = u(1) = 0 \quad \text{(Dirichlet BCs)}$$

Exact solution for $f(x) = \pi^2 \sin(\pi x)$:

$$u^\star(x) = \sin(\pi x)$$

QPINN residual loss:

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{k=1}^{N_r} \left[-\frac{d^2u_\theta}{dx^2}\bigg|_{x_k} - f(x_k) \right]^2$$

Boundary loss:

$$\mathcal{L}_b = u_\theta(0)^2 + u_\theta(1)^2$$

Second derivative via input-shift:

$$\frac{d^2u_\theta}{dx^2}\bigg|_x = \frac{1}{2}\left[u_\theta(x+\tfrac{\pi}{2}) - 2u_\theta(x) + u_\theta(x-\tfrac{\pi}{2})\right]$$

This uses the identity for $u_\theta(x) = \langle O \rangle_x$ with $S(x) = e^{-ixP/2}$:

$$\partial_{xx}\langle O\rangle_x = -\langle O\rangle_x + \tfrac{1}{2}\left(\langle O\rangle_{x+\pi/2} + \langle O\rangle_{x-\pi/2}\right)$$

Three circuit evaluations

The Laplacian at one point costs exactly 3 circuit runs: $x - \pi/2$, x, $x + \pi/2$.

Variational Formulation and Connection to FEM

The Poisson equation has an equivalent **variational form**:

$$\min_u \mathcal{E}[u] = \int_0^1 \left[\tfrac{1}{2}(u')^2 - f\,u\right] dx$$

subject to $u(0) = u(1) = 0$.
The QPINN can be trained directly on the *energy functional* (Ritz method):

$$\mathcal{L}_{\text{Ritz}}(\theta) = \sum_k w_k \left[\tfrac{1}{2} \left(\left. \frac{du_\theta}{dx} \right|_{x_k} \right)^2 - f(x_k)\, u_\theta(x_k) \right]$$

where w_k are quadrature weights.

Why Ritz can outperform strong form

- Needs only first derivatives (one input-shift per point)
- Smoother loss landscape
- Natural for symmetric positive-definite operators
- Directly minimises the physical energy

Connection to FEM

The Ritz-QPINN is a *quantum Galerkin method*: the quantum ansatz $u_\theta(x)$ plays the role of the finite element basis, but spans an exponentially large function space.

Generalisation: ODEs and Other PDEs

1D harmonic oscillator ODE:

$$\frac{d^2u}{dx^2} + \omega^2 u = 0$$

Residual: $r_k = u''_\theta(x_k) + \omega^2 u_\theta(x_k)$

1D heat equation (time-dependent):

$$\partial_t u - \kappa \partial_{xx} u = 0$$

Encode (x, t) into circuit:
$S(x,t) = R_y(\pi x) \otimes R_y(\pi t)$; evaluate time and space derivatives by input-shift on respective qubit registers.

Burgers equation (nonlinear):

$$\partial_t u + u\,\partial_x u = \nu\,\partial_{xx} u$$

Nonlinear term $u\partial_x u$: product of two circuit outputs

Schrödinger equation as a QPINN

$$i\partial_t \psi = -\tfrac{1}{2}\partial_{xx}\psi + V(x)\psi$$

The quantum ansatz $\psi_\theta(x,t)$ has a *natural* complex structure: measure real and imaginary parts via two different observables (Z and Y).
The QPINN loss enforces the time-evolution equation directly — a bridge between QPINNs and Hamiltonian simulation.

Key point

All linear differential operators act linearly on the circuit output $u_\theta(x)$, so residuals always reduce to sums of circuit evaluations at shifted inputs.

Unified Shift Framework: Parameters and Inputs

For a quantum circuit with a gate $e^{-i\phi P/2}$ (Pauli P, half-angle ϕ):

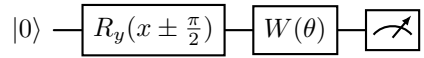
$$\frac{\partial}{\partial\phi}\langle O\rangle_\phi = \frac{1}{2}\left[\langle O\rangle_{\phi+\pi/2} - \langle O\rangle_{\phi-\pi/2}\right]$$

This formula is universal:

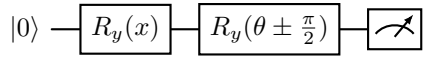
- $\phi = \theta_\mu$ (variational parameter): **parameter-shift rule** — gives gradient w.r.t. trainable weights
- $\phi = x$ (input encoding): **input-shift rule** — gives spatial derivative of the PDE solution

Higher-order derivatives by repeated application:

$$\partial_\phi^n\langle O\rangle = \sum_{k=0}^{n}(-1)^k\binom{n}{k}\langle O\rangle_{\phi+(\frac{n}{2}-k)\frac{\pi}{2}}$$



Input-shift: evaluates $\partial_x u_\theta$ using 3 circuit runs at $x-\frac{\pi}{2}$, x, $x+\frac{\pi}{2}$.



Parameter-shift: evaluates $\partial_\theta u_\theta$ using 2 circuit runs at $\theta\pm\frac{\pi}{2}$.

Total budget

k-th order spatial derivative at one point: $k+1$ circuit runs.

Commutation of Differential Operators with Circuit Output

Let $u_\theta(x) = \langle 0| U^\dagger(x,\theta) O\, U(x,\theta) |0\rangle$. For a *linear* differential operator $\mathcal{D}$:

$$\mathcal{D}[u_\theta](x) = \langle 0| U^\dagger(x,\theta)\, \tilde{O}(x,\mathcal{D})\, U(x,\theta) |0\rangle$$

where $\tilde{O}(x,\mathcal{D})$ is the *effective observable* induced by $\mathcal{D}$ acting on $U(x,\theta)$.
For $\mathcal{D} = \partial_x^2$:

$$\partial_{xx} u_\theta(x) = \langle 0| \partial_{xx} \left[U^\dagger O\, U\right] |0\rangle = \langle 0| \tilde{O}_{xx} |0\rangle$$

The effective observable $\tilde{O}_{xx}$ is:

$$\tilde{O}_{xx} = U^\dagger(\partial_{xx} U) O + 2U^\dagger(\partial_x U) O (\partial_x U^\dagger) U + \cdots$$

Practical evaluation

Rather than computing $\tilde{O}_{xx}$ analytically, the input-shift rule evaluates it implicitly:

$$\partial_{xx} u_\theta(x) \approx \frac{u_\theta(x+h) - 2u_\theta(x) + u_\theta(x-h)}{h^2}$$

with $h = \pi/2$ for *exact* (not approximate) results.

Linearity is key

For *linear* PDEs, the QPINN gradient decomposes into a sum of parameter-shift evaluations, each at a different input shift — no cross-terms. Nonlinear PDEs require products of circuit outputs.

Expressivity of QPINNs: Fourier Analysis

Theorem (Schuld et al. 2021). The output of any QNN with data-encoding gates $e^{-ixP/2}$ (Pauli P) is a *partial Fourier series*:

$$u_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta)\, e^{i\omega x}$$

where the set of accessible frequencies is:

$$\Omega \subseteq \left\{ \omega_1 + \omega_2 + \cdots + \omega_n : \omega_i \in \{0, \pm\tfrac{1}{2}\} \right\}$$

The coefficients $c_\omega(\theta)$ depend on the variational parameters; the frequencies Ω depend only on the *encoding*.

For solving PDEs, the highest frequency in Ω limits the shortest spatial scale that can be resolved.

Extending the frequency spectrum

- More qubits $\rightarrow$ denser Ω
- Data re-uploading $\rightarrow$ integer multiples
- IQP encoding $\rightarrow$ pairwise frequencies $x_i x_j$

Consequence for PDEs

A QPINN with n qubits and L re-uploading layers can represent solutions containing frequencies up to $\omega_{\max} = Ln/2$. For a target function with frequencies up to $\omega^\star$, we need:

$$Ln \geq 2\omega^\star$$

qubits $\times$ layers.

Approximation Error and Convergence

Let $u^\star \in H^s(\Omega)$ (Sobolev space of order s) be the exact PDE solution. The QPINN approximation error decomposes into three parts:

$$\|u_{\theta^\star} - u^\star\|_{L^2} \leq \underbrace{\varepsilon_{\text{approx}}}_{\text{expressivity}} + \underbrace{\varepsilon_{\text{opt}}}_{\text{optimisation}} + \underbrace{\varepsilon_{\text{stat}}}_{\text{sampling}}$$

Approximation error (best possible):

$$\varepsilon_{\text{approx}} \sim \|\hat{u}^\star_\omega\| \text{ for } \omega \notin \Omega$$

Controlled by the frequency spectrum of $u^\star$ and Ω.
Optimisation error: depends on the loss landscape (barren plateaus, local minima).
Statistical error: $\varepsilon_{\text{stat}} \sim N_r^{-1/2}$ from finite collocation points.

No free lunch

QPINNs do not automatically outperform classical PINNs for arbitrary PDEs.
Advantage requires:

1. Solutions with quantum-compressible frequency structure
2. High-dimensional problems ($d \gg 1$) where quantum memory helps
3. Availability of quantum hardware

High-d advantage

For d-dimensional PDEs, the classical PINN needs $O(\varepsilon^{-d})$ parameters; the QPINN can compress the solution using $O(d)$ qubits if the solution has a tensor-product structure

Barren Plateaus in QPINNs and Mitigation

QPINNs suffer from the same barren plateau phenomenon as general QNNs (from week15.tex):

$$\mathrm{Var}\left(\frac{\partial \mathcal{L}}{\partial \theta_\mu}\right) \sim \frac{1}{2^n}$$

Additional sources in QPINNs:

- Residual $r_k \approx 0$ early in training: gradient is small even without barren plateaus
- Loss balance: $\mathcal{L}_r \gg \mathcal{L}_b$ makes BC enforcement slow
- Stiff PDEs: widely varying eigenvalues of the differential operator

Mitigation strategies for QPINNs

1. **Physics-inspired ansatz:** layer structure mirrors the Green's function of the differential operator
2. **Incremental training:** start with low-frequency modes, progressively add higher frequencies
3. **Adaptive collocation:** concentrate points where residual is large
4. **Loss weighting:** normalise $\mathcal{L}_r$ and $\mathcal{L}_b$ by their gradient norms
5. **Natural gradient (QFI):**

$$\dot{\theta} = F^{-1} \nabla_\theta \mathcal{L}$$

Respects the geometry of the quantum

QPINN vs. Classical PINN: Comparison

Feature	Classical PINN	QPINN
Ansatz	Deep MLP $u_\theta(x)$	Circuit output $\langle O \rangle_{x,\theta}$
Derivatives	Automatic differentiation	Input-shift rule (exact)
Gradient	Backpropagation	Parameter-shift rule (exact)
Memory	$O(N_p)$ parameters	2^n amplitudes, $O(n)$ params
Freq. spectrum	Arbitrary (but spectral bias)	Bounded by encoding
High-d PDE	Exponential cost	Potential $O(d)$ qubit advantage
Hardware	GPU/CPU	Quantum device or simulator
Maturity	Production-ready	Research / NISQ

When QPINNs are most promising

High-dimensional PDEs; solutions with structured frequency spectra; problems with natural quantum simulation analogues (e.g., Schrödinger, Maxwell).

Python Implementation: Gates and Circuit

```python
import numpy as np
from scipy.optimize import minimize

# -- Gate definitions ------------------------------------------
def ry(t):
    c, s = np.cos(t / 2), np.sin(t / 2)
    return np.array([[c, -s], [s, c]], dtype=complex)

def rz(t):
    return np.diag([np.exp(-1j * t / 2), np.exp(1j * t / 2)]).astype(complex)

I2   = np.eye(2, dtype=complex)
CNOT = np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]], dtype=complex)
Z0   = np.kron(np.diag([1., -1.]).astype(complex), I2)  # Z on qubit 0

def var_layer(p):
    """Ry(p0)xRy(p1) -> CNOT -> Rz(p2)xRz(p3)"""
    return np.kron(rz(p[2]), rz(p[3])) @ CNOT @ np.kron(ry(p[0]), ry(p[1]))

def circuit(x, params):
    """
    2-qubit QPINN circuit for 1D PDE.
    Encoding: Ry(pi*x) on each qubit.
    Ansatz:   2 variational layers (8 parameters).
    Output:   <Z_0> in [-1, 1].
    """
```

Derivatives via Input-Shift Rule

```python
# -- Input-shift rule for spatial derivatives ----------------------
SHIFT = np.pi / 2   # shift for Pauli Ry generator

def du_dx(x, params):
    """First derivative d/dx u_theta(x) via input-shift rule."""
    return 0.5 * (circuit(x + SHIFT, params)
                - circuit(x - SHIFT, params))

def d2u_dx2(x, params):
    """
    Second derivative d^2/dx^2 u_theta(x) via second-order shift.
    Uses: f''(x) = f(x + pi/2) - 2*f(x) + f(x - pi/2)
    (exact for generators with eigenvalues +/- 1/2).
    """
    return (circuit(x + SHIFT, params)
          - 2 * circuit(x, params)
          + circuit(x - SHIFT, params))

# -- Source term for Poisson: f(x) = pi^2 * sin(pi*x) ------------
def source(x):
    return np.pi**2 * np.sin(np.pi * x)

# -- Exact solution for validation -------------------------------
def exact(x):
    return np.sin(np.pi * x)
```

QPINN Loss Function and Gradient

```python
# -- Collocation and boundary points ------------------------------
np.random.seed(42)
N_r = 20    # interior collocation points
N_b = 2     # boundary points (x=0 and x=1)
x_r = np.linspace(0.05, 0.95, N_r)    # interior
x_b = np.array([0.0, 1.0])            # boundary
lambda_bc = 10.0                       # BC weight

# -- QPINN loss ---------------------------------------------------
def loss(params):
    # PDE residual: -u''(x) - f(x) = 0
    residuals = np.array([
        -d2u_dx2(x, params) - source(x) for x in x_r
    ])
    L_r = np.mean(residuals**2)

    # Boundary conditions: u(0) = u(1) = 0
    L_b = circuit(x_b[0], params)**2 + circuit(x_b[1], params)**2

    return L_r + lambda_bc * L_b

# -- Parameter-shift gradient (exact) -----------------------------
def grad_loss(params, shift=np.pi / 2):
    g = np.zeros(len(params))
    for mu in range(len(params)):
        p_plus  = params.copy(); p_plus[mu]  += shift
```

Training Loop and Results

```python
# -- Training ------------------------------------------------------
np.random.seed(7)
params = np.random.uniform(0, np.pi, 8)   # 2 layers x 4 params

history = []
# Adam hyperparameters
lr, b1, b2, eps = 0.15, 0.9, 0.999, 1e-8
m, v = np.zeros(8), np.zeros(8)

for step in range(300):
    g = grad_loss(params)
    t = step + 1
    m = b1 * m + (1 - b1) * g
    v = b2 * v + (1 - b2) * g**2
    mc, vc = m / (1 - b1**t), v / (1 - b2**t)
    params -= lr * mc / (np.sqrt(vc) + eps)
    history.append(loss(params))
    if (step + 1) % 75 == 0:
        print(f"Step {step+1:4d}  loss = {history[-1]:.6f}")

# -- Evaluate and compare to exact solution ----------------------
x_test   = np.linspace(0, 1, 100)
u_qpinn  = np.array([circuit(x, params) for x in x_test])
u_exact  = exact(x_test)
l2_error = np.sqrt(np.mean((u_qpinn - u_exact)**2))
print(f"\nL2 error vs. sin(pi*x): {l2_error:.4f}")
```

Plotting and Interpretation

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))

# -- Solution comparison ----------------------------------------
axes[0].plot(x_test, u_exact, 'k-', lw=2, label=r'Exact $\sin(\pi x)$')
axes[0].plot(x_test, u_qpinn, 'b--', lw=2, label='QPINN (2 qubits, 2 layers)')
axes[0].scatter(x_r, np.zeros_like(x_r), s=20, c='r', zorder=5,
                label='Collocation points')
axes[0].set_xlabel('$x$'); axes[0].set_ylabel('$u(x)$')
axes[0].set_title('QPINN solution: $-u\'\'=\\pi^2\\sin(\\pi x)$')
axes[0].legend(fontsize=8)

# -- Training loss ----------------------------------------------
axes[1].semilogy(history, 'steelblue')
axes[1].set_xlabel('Training step'); axes[1].set_ylabel('Loss')
axes[1].set_title('QPINN training loss (Adam, lr=0.15)')

plt.tight_layout(); plt.show()
```

Expected output:

- L2 error $\lesssim 0.05$ after 300 steps
- Loss decreases by 2–3 orders of magnitude

Improving accuracy:

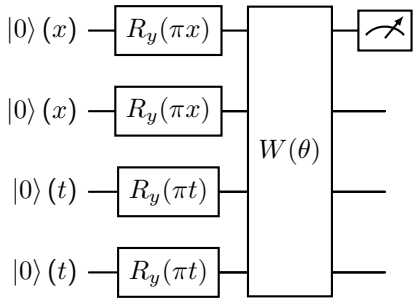
- More qubits (wider frequency spectrum)
- More variational layers

Extension: Time-Dependent PDEs

For $u(x,t)$, encode both space and time:

$$S(x,t) = R_y(\pi x) \otimes R_y(\pi t) \otimes \cdots$$

1D heat equation: $\partial_t u - \kappa \partial_{xx} u = 0$
QPINN circuit on $n_x + n_t$ qubits:



Input shifts for PDE residual:

$$\partial_t u \approx \tfrac{1}{2}[u(x,t+\tfrac{\pi}{2}) - u(x,t-\tfrac{\pi}{2})]$$
$$\partial_{xx} u \approx u(x+\tfrac{\pi}{2},t) - 2u(x,t) + u(x-\tfrac{\pi}{2},t)$$

Each point: 5 circuit evaluations for residual.

Initial condition

Add IC loss:

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_x} \sum_k \left| u_\theta(x_k, 0) - u_0(x_k) \right|^2$$

Scaling

d-dimensional PDE: $n_x + n_t = n$ qubits.
Classical PINN: exponential in n; QPINN: linear. Advantage is largest when $d \gg 1$.

The Schrödinger Equation as a QPINN

The time-dependent Schrödinger equation:

$$i\hbar\,\partial_t\psi(x,t) = H\psi(x,t), \quad H = -\frac{\hbar^2}{2m}\partial_{xx} + V(x)$$

Split into real and imaginary parts: $\psi = \psi_R + i\psi_I$:

$$\begin{aligned}\partial_t\psi_R &= \ \ \frac{\hbar}{2m}\partial_{xx}\psi_I - V\psi_I/\hbar \\ \partial_t\psi_I &= -\frac{\hbar}{2m}\partial_{xx}\psi_R + V\psi_R/\hbar\end{aligned}$$

Two QPINNs: $\psi_R \approx \langle Z_0\rangle_{x,t,\theta_R}$ and $\psi_I \approx \langle Z_0\rangle_{x,t,\theta_I}$.

Residual losses

$$r_R = \partial_t\psi_R - \tfrac{\hbar}{2m}\partial_{xx}\psi_I + \tfrac{V}{\hbar}\psi_I$$

$$r_I = \partial_t\psi_I + \tfrac{\hbar}{2m}\partial_{xx}\psi_R - \tfrac{V}{\hbar}\psi_R$$

All derivatives by input-shift; $V(x)$ evaluated analytically.

Bridge to Hamiltonian simulation

The QPINN Schrödinger formulation is the classical simulation of quantum time evolution. On a quantum device, Hamiltonian simulation provides $e^{-iHt/\hbar}$ exactly — the QPINN bridges continuous PDE solving and digital quantum simulation.

Application to Eigenvalue Problems

The time-independent Schrödinger equation:

$$H\psi(x) = E\psi(x)$$

is a **quantum eigenvalue problem**. A QPINN approach minimises the Rayleigh quotient:

$$E[\psi_\theta] = \frac{\langle\psi_\theta|H|\psi_\theta\rangle}{\langle\psi_\theta|\psi_\theta\rangle} \geq E_0$$

Connection to VQE (week15.tex):
This is precisely the VQE objective, with the quantum ansatz encoding the spatial wavefunction $\psi_\theta(x)$. The QFI metric governs the geometry of the wavefunction manifold; TDVP (week15.tex) provides the optimal projection of the Schrödinger equation.

Excited states

Minimise the augmented loss:

$$\mathcal{L}_n(\theta) = \langle H\rangle_\theta + \sum_{k<n} \mu_k |\langle\psi_k|\psi_\theta\rangle|^2$$

Penalty μ_k enforces orthogonality to the already-found states $\psi_0, \ldots, \psi_{n-1}$. QPINN naturally extends to excited states of the Schrödinger, Helmholtz, and Sturm-Liouville equations.

QPINN $\leftrightarrow$ VQE

VQE on a qubit Hamiltonian (week15.tex) is the *discretised* version of a QPINN applied to the Schrödinger equation on a spatial grid.

Exercises

1. **Harmonic oscillator.** Modify the Poisson code to solve $u'' + \omega^2 u = 0$ with $u(0) = 0$, $u'(0) = 1$. Compare to the exact solution $u(x) = \sin(\omega x)/\omega$.
2. **Second-order input-shift identity.** Prove that for $u_\theta(x) = \langle O \rangle$ with $S(x) = e^{-ixP/2}$ (Pauli P, eigenvalues $\pm\frac{1}{2}$):
$$\frac{d^2 u_\theta}{dx^2} = u_\theta(x + \tfrac{\pi}{2}) - 2u_\theta(x) + u_\theta(x - \tfrac{\pi}{2})$$
3. **Frequency analysis.** For a 2-qubit QPINN with angle encoding $R_y(\pi x)$ on each qubit and $L = 2$ variational layers, list all accessible Fourier frequencies $\omega \in \Omega$. What is the highest frequency representable?
4. **Ritz method.** Implement the variational (Ritz) QPINN loss $\mathcal{L}_{\text{Ritz}} = \sum_k w_k [\frac{1}{2}(u'_\theta)^2 - f u_\theta]$ for the Poisson problem and compare convergence to the strong-form residual loss.
5. **Heat equation.** Extend the code to solve $\partial_t u = \partial_{xx} u$, $x \in (0, 1)$, $t \in (0, 0.5)$, with $u(x, 0) = \sin(\pi x)$ and $u(0, t) = u(1, t) = 0$. Exact solution: $u(x, t) = e^{-\pi^2 t} \sin(\pi x)$.

Outlook: Open Problems and Future Directions

Near-term (NISQ)

- Small QPINNs for 1D/2D PDEs on simulators
- Error mitigation for noisy derivatives
- Hybrid: QPINN ansatz + classical finite element solver
- Benchmarking vs. classical PINNs on spectral-bias-prone problems

Fundamental trade-off

$$\underbrace{n \text{ qubits}}_{\text{exp. memory}} \longleftrightarrow \underbrace{2^n \text{ amplitudes}}_{\text{hard to read out}}$$

Read-out bottleneck may eliminate quantum advantage.

Long-term directions

- **Quantum simulation of PDEs:** Hamiltonian simulation provides e^{-iHt} exactly — QPINNs as a variational bridge
- **High-d PDEs:** molecular dynamics, financial derivatives, kinetic theory
- **Inverse problems:** learn the coefficients of a PDE from data (combination of QPINN + QBM)
- **Multi-physics:** coupled PDE systems via multi-register quantum circuits
- **Error correction:** fault-tolerant QPINNs for provably accurate PDE solvers

Summary: QPINN in Context

	Classical PINN	**QPINN**
Ansatz	MLP $u_\theta(x)$	Circuit $\langle O \rangle_{x,\theta}$
Derivatives	Autograd	Input-shift (exact)
Parameter grad	Backprop	Parameter-shift (exact)
Frequency	Spectral bias	Bounded, explicit Ω
Memory	$O(N_p)$	$O(n)$ params, 2^n states
BC enforcement	Loss penalty	Same
Optimiser	Adam, L-BFGS	Adam + natural gradient
Geometry	Euclidean	QFI / Fubini-Study
PDE target	Any	Linear PDEs best
Advantage	Production-ready	High-d, quantum PDEs

The central message

References

Classical PINNs

1. M. Raissi, P. Perdikaris, G. E. Karniadakis, *Physics-informed neural networks*, J. Comput. Phys. **378**, 686 (2019).
2. S. Wang, X. Yu, P. Perdikaris, *When and why PINNs fail to train*, J. Comput. Phys. **449**, 110768 (2022).

Quantum PINNs

3. A. Kyriienko, A. E. Paine, V. E. Elfving, *Solving nonlinear differential equations with differentiable quantum circuits*, Phys. Rev. A **103**, 052416 (2021).
4. T. Goto et al., *Universal approximation property of quantum machine learning models in quantum-enhanced feature spaces*, Phys. Rev. Lett. **127**, 090506 (2021).

Fourier analysis of QNNs

5. M. Schuld et al., *Effect of data encoding on the expressive power of variational quantum machine learning models*, Phys. Rev. A **103**, 032430 (2021).

Data re-uploading and expressivity

6. A. Pérez-Salinas et al., *Data re-uploading for a universal quantum classifier*, Quantum **4**, 226 (2020).
7. H.-Y. Huang et al., *Power of data in quantum machine learning*, Nat. Commun. **12**, 2631 (2021).

QNN foundation (see also week15.tex)

8. J. Stokes et al., *Quantum natural gradient*, Quantum **4**, 269 (2020).
9. M. Cerezo et al., *Variational quantum*